Technical Documentation: RAG Customer Support Assistant

1. Introduction

Retrieval-Augmented Generation (RAG) is a technique that grounds large language models (LLMs) on verifiable, external data sources. In enterprise environments, LLMs often hallucinate or fail to answer domain-specific questions because they lack internal context. Applying RAG mitigates this by fetching explicit facts from an internal corpus before synthesizing the prompt.

This specific project applies RAG to a Customer Support automation use-case. By feeding the system operational manuals and FAQs, it acts as a highly knowledgeable, instantaneous tier-1 support agent, actively routing nuanced escalations to a human supervisor.

2. System Architecture Explanation

The architecture is broadly split into two distinct pipelines: The Ingestion Pipeline and the Generation Workflow.

- **Ingestion**: Raw PDFs are destructured into plaintext. A chunking pipeline normalizes text boundaries because LLMs and embedders have context-window limits. The chunks are converted into geometrical vectors and housed in ChromaDB.
- **Generation Workflow (LangGraph)**: An event loop acts upon user inputs. A routing node predicts if the query implies frustration, urgency, or extreme complexity. If standard, it requests ChromaDB for context, injects the response into an LLM prompt, and yields an answer. If risky, it delegates to a manual process.

3. Design Decisions

- **Chunk Size / Overlap**: Opted for ~1000 characters with 150-character overlaps. If chunks are too small, paragraphs lose meaning (e.g., separating a "Step 1" from "Step 2"). If too large, irrelevancy dilutes the LLM prompt. The overlap ensures continuity between semantic borders.
- **Embedding Strategy**: Using `all-MiniLM-L6-v2` or similar dense passage retrievers balance computation cost against semantic accuracy.
- **Retrieval Approach**: Standard Cosine Similarity $Top\text{-}k$ retrieval ($k=4$). This returns enough diverse information without flooding the LLM.
- **Prompt Design**: Framed explicitly: "You are a customer support agent. Answer the following user question strictly based on the provided context. If the context does not contain the answer, reply that you don't know."

4. Workflow Explanation (LangGraph)

LangGraph's core advantage is its formalization of loops and state machines over static DAGs (Directed Acyclic Graphs).

- **State**: A common TypedDict that flows through all nodes.

- **Intent Node**: A fast, deterministic router. Based on the state's `question`, it updates `escalation_required`.
- **Retrieve Node**: Connects to the local vectorstore. Mutates the state array `documents`.
- **Generate Node**: Mutates the state string `generation`.

5. Conditional Logic

The routing decision determines the trajectory from the Intent Node. If `escalation_required == True`, a ConditionalEdge routes the data to a manual supervisor node. Otherwise, it follows the data to the Retreiver node. Intent detection can be implemented flexibly: either via a specialized LLM zero-shot classification call or programmatic heuristic checks against angry/frustrated keyword lists.

6. HITL Implementation

Human-In-The-Loop prevents AI systems from operating autonomously when dealing with high-risk user frustrations.

- **Intervention**: By defining `interrupt_before=["human_approval"]` in LangGraph, the execution thread halts exactly at the boundary. The in-memory SQLite checkpointer persists the thread state.
- **Benefits**: Ensures brand safety and handles complex billing/security scenarios standard AI cannot legally process.
- **Limitations**: Creates potential bottlenecks during peak hours if agents aren't available to clear the queue.

7. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed**: Exhaustive cross-encoder re-ranking pipelines improve context quality dramatically, but increase latency. We rely on fast, bi-encoder similarity metrics for real-time chat responsiveness.
- **Cost vs. Performance**: Hosted foundational models (GPT-4) provide impeccable reasoning but scale costly. Utilizing smaller, specialized local models via Ollama or HuggingFace optimizes TCO at the expense of infrastructure management overhead.

8. Testing Strategy

1. **Unit Testing**: Isolated tests for parsing (ensuring text output holds no artifact formatting) and chunk overlaps.
2. **Integration Testing**: Verifying complete Graph execution paths using predefined queries:

    - *Example 1 (Standard)*: "What is your return policy window?" -> Should traverse Intent -> Retrieve -> Generate.
    - *Example 2 (HITL)*: "I am extremely angry and want to speak to a manager!" -> Should traverse Intent -> HITL.

9. Future Enhancements

- **Multi-Document Support**: Introduce namespace routing in ChromaDB to distinguish

between 'Billing', 'TechSupport', and 'Sales' documents to reduce context collision.
- **Feedback Loop Integration**: Storing positive/negative user reactions to incrementally fine-tune the Generation prompt or embedding space mapping.
- **Memory Integration**: Leveraging LangGraph's native thread memory to support multi-turn, contextual conversations.